

# SpaTE: Learning-Based Traffic Engineering under Sparse Traffic Observation in LEO Satellite Constellations

Linhui Wei, He Ma, Yumei Wang, *Member, IEEE*, and Guangteng Fan

**Abstract**—Low-Earth-orbit (LEO) satellite constellations require traffic engineering (TE) that reacts quickly to changing topology and demand. Many learned TE systems achieve low online latency but assume a complete traffic matrix, whereas telemetry and onboard-resource constraints allow a constellation controller to meter only a fraction of demands at each epoch. This paper presents SpaTE, a sparse-aware TE system that allocates all demands directly from incomplete observations, and rests on three ideas. First, each unmeasured demand is represented by a learnable mask embedding, so the model input separates “not measured” from “no traffic” instead of fabricating a volume. Second, message passing over the path-link graph is gated asymmetrically: unmeasured demands read link states but write nothing into them, which provably keeps surrogate values out of the link representation and out of every measured demand’s allocation. Third, a temporal encoder over recent sparse snapshots is fused with the spatial representation by per-demand cross-attention, so history governs which spatial evidence to trust where the current entry is missing. A source-neighbor estimate additionally supplies the congestion signal for optional deployment-time refinement without access to the ground-truth matrix. On a 1,584-satellite Starlink constellation with 2%–50% observability, SpaTE achieves a mean maximum link utilization (MLU) of 1.64–1.89 with sub-0.4-s decisions. Comparisons with imputation-based and temporal controls do not establish a statistically significant overall advantage. Ablations show a 6.7% mean MLU reduction from training at 50% observability, while gains remain dependent on the observation ratio.

**Index Terms**—LEO satellite constellation, traffic engineering, sparse traffic measurement, maximum link utilization (MLU), graph neural networks.

## 1 INTRODUCTION

LOW-EARTH-ORBIT (LEO) satellite constellations are rapidly scaling toward tens of thousands of satellites [1], [2]. Their inter-satellite links (ISLs) carry spatially concentrated and temporally varying traffic under limited capacity [3]. Traffic engineering (TE), which coordinates how each demand is split across candidate paths, is therefore a key control primitive [4], [5]. Classical optimization can take minutes at constellation scale, whereas learned allocators reduce online decision time to the sub-second range by moving most computation offline [6], [7], [8].

Many learned TE systems nevertheless rely on a complete traffic matrix at every control epoch. This assumption is particularly fragile in an LEO constellation. Measuring every demand requires satellites to meter, aggregate, and transmit fine-grained statistics through telemetry channels that compete with user traffic for ISL and ground-station bandwidth [3], [9]. It also consumes onboard computation, storage, and power, while intermittent ground contacts introduce unavoidable gaps; the expense of network-wide metering is the very reason terrestrial operators have long inferred traffic matrices instead of measuring them [10], [11]. As illustrated in Fig. 1, only a subset of satellites may meter

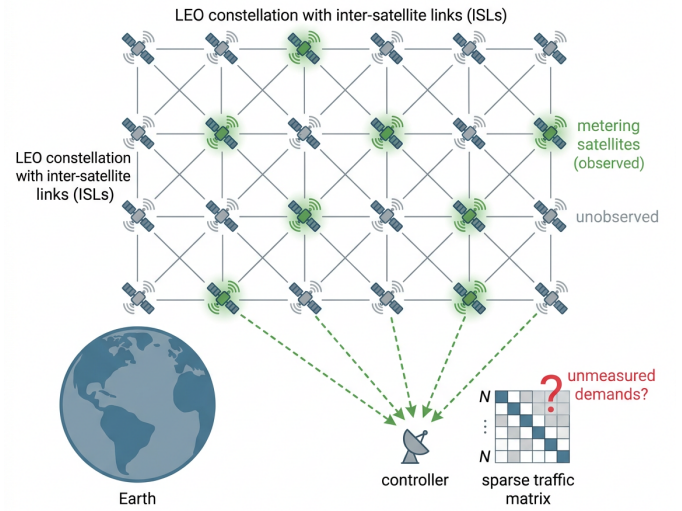


Fig. 1. Sparse traffic observation in an LEO constellation. Only a subset of satellites meters traffic, so the controller receives a sparse traffic matrix while remaining responsible for all demands.

- L. Wei and G. Fan are with the National Innovation Institute of Defense Technology, Academy of Military Science, Beijing, China (e-mail: weilinhui998@163.com; fanguangteng@163.com).
- H. Ma and Y. Wang are with the School of Artificial Intelligence, Beijing University of Posts and Telecommunications, Beijing, China (e-mail: mahe@bupt.edu.cn; ymwang@bupt.edu.cn).
- Manuscript received XXX; revised XXX. (Corresponding author: Guangteng Fan.)

traffic at a given epoch. The controller then receives a sparse traffic matrix but must still allocate every demand. Efficient TE from sparse observations is thus a natural operating mode for LEO networks, not merely a degraded version of complete measurement.

In LEO constellations, sparse observation interacts with

geographic traffic concentration and orbital motion in three ways. First, when metering is available on only a subset of satellites, an unmeasured source hides a whole row of demands, not just isolated matrix entries. Since satellites serving populated footprints can carry substantial traffic [3], zero filling may erase an entire source's contribution to ISL load; explicit matrix completion instead risks passing reconstruction errors into routing [12]. Second, traffic from nearby footprints shares multi-hop ISL corridors, while satellite motion changes the association between ground users and ingress satellites. The allocator must combine current path-link relationships with recent traffic evolution without allowing imputed values to distort shared link representations [13]. Third, moving footprints change the active source-destination pairs as the constellation evolves. A model tied to a fixed demand list would require resizing or retraining, despite the limited onboard resources and short control windows [8]. These coupled measurement and mobility constraints distinguish sparse-observation TE in LEO constellations from the static terrestrial setting [14].

We present SpaTE (Sparse-aware Traffic Engineering), a learned allocator designed around these constraints. It represents each unobserved demand with a learnable mask embedding rather than a fabricated volume. A mask-gated flow-centric graph neural network (GNN) lets unobserved demands receive link-state information without injecting their *placeholders*, the surrogate values that stand in for unmeasured demands at the model input, into link embeddings. A temporal Transformer [15] encodes recent sparse observations, and per-demand cross-attention fuses temporal and spatial evidence before predicting path split ratios. Shared weights and demand pruning keep the model independent of the instantaneous demand set, enabling reuse across constellation snapshots and scales.

After allocation, SpaTE can optionally refine residual overloads using link loads estimated from the same sparse observation. The important input to this lightweight step is a neighbor estimate: an unmeasured demand is inferred from observed demands sharing its source satellite. This estimate uses geographic locality to avoid granting the controller an unavailable ground-truth traffic matrix. It is an implementation aid to the sparse-aware allocator rather than the organizing principle of the system.

We evaluate SpaTE on a 1,584-satellite Starlink Shell-1 constellation over observation ratios from 2% to 50%, measuring the realized maximum link utilization (MLU) against ground-truth demands. Each configuration is evaluated through repeated runs with matched random initializations, separating observation-dependent effects from variation between runs. SpaTE holds the realized MLU in the 1.64–1.89 range, within  $1.19\text{--}1.37\times$  of a path-restricted LP optimum, while deciding in under 0.4 s per snapshot and transferring to a one-third-scale constellation without architectural change. Against impute-then-optimize pipelines and zero-filled temporal baselines the outcomes are mixed and no single pairwise comparison reaches significance; we therefore examine the distributions across repeated runs to identify where sparse-aware allocation helps and where simple fill-based treatments remain competitive.

Our contributions are as follows:

- We formulate sparse-observation TE for LEO constellations in which limited satellite telemetry leaves individual demands or entire source rows unmeasured. The formulation separates the controller's partial information from the true ISL loads used to evaluate its decisions. Predicting path split ratios allows routing decisions for unmeasured demands without requiring explicit recovery of their volumes.
- We design SpaTE to combine the spatial structure of ISL routes with the recent evolution of geographically concentrated satellite traffic. Learnable mask embeddings distinguish missing measurements from zero traffic, asymmetric path-link propagation isolates those embeddings from measured link evidence, and cross-attention integrates the spatial features with sparse observation history. Demand pruning and shared parameters let the same architecture accommodate changing active pairs as satellite footprints move, without tying its weights to a fixed constellation graph.
- We evaluate SpaTE on a 1,584-satellite constellation across 2%–50% observability, with a smaller same-family constellation as a scale check. Matched comparisons and ablations distinguish the effects of sparse-input representation, training, and observation-based refinement. The results identify observation-dependent performance and initialization sensitivity, bounding the conditions under which the proposed design improves MLU instead of assuming a uniform advantage.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 formulates the sparse-observation TE problem. Section 4 presents SpaTE. Section 5 evaluates the system, and Section 6 concludes the paper.

## 2 RELATED WORK

### 2.1 ML-Driven Traffic Engineering for WANs

Traffic engineering has long been formulated as a constrained optimization problem [16], [17] solved by linear programming (LP), whose runtime scales poorly with network size. Production software-defined WANs centralize this computation to raise utilization [4], [18], [5]. Kumar *et al.* proposed SMORE [19], which decouples robust oblivious path selection from online rate adaptation, while Abuzaid *et al.* [20] and Narayanan *et al.* [21] accelerate LP by problem decomposition, trading allocation quality for speed. A recent line of work replaces online solvers with learned models: Valadarsky *et al.* [22] learn edge weights from traffic history and obtain forwarding splits through softmax routing. Perry *et al.* proposed DOTE [7], which directly maps historical traffic matrices to routing splits with stochastic optimization, without explicitly predicting a future matrix. These are predictive TE methods: historical measurements must not be confused with knowledge of the complete target-time traffic matrix. Xu *et al.* proposed Teal [6], combining a flow-centric graph neural network (GNN), multi-agent reinforcement learning (RL), and alternating direction method of multipliers (ADMM) [23] fine-

tuning to achieve near-optimal allocations with orders-of-magnitude speedups; and Liu *et al.* proposed FIGRET [24], which enhances robustness against traffic uncertainty at per-flow granularity. Graph models have also been used to predict network behavior rather than allocate directly, as in RouteNet [25]. To survive topology changes without retraining, AlQiam *et al.* proposed HARP [26], enforcing invariances to node permutation and tunnel reordering so that it transfers across topologies unseen in training; Fan *et al.* proposed Aether [27], pursuing the same generality with elastic multi-agent graph transformers, and Ye *et al.* [28] trained path-based GNNs for robust distributed TE under link failures. More recently, Wu *et al.* [29] explored large language models as general-purpose traffic planners. Full-matrix allocation and historical-input prediction have different information requirements. In particular, adapting DOTE or historical-input RL to unmeasured entries requires an explicit sparse-input protocol; natural zero traffic is not the same as an unavailable measurement. Comparisons must align the observation mask and the latest input time, rather than give one method current measurements while restricting another to past measurements.

## 2.2 TE and Routing for Satellite Networks

LEO mega-constellations pose unique challenges for TE: topologies change every few tens of milliseconds as satellites move, and control decisions must be produced within stringent latency budgets [1], [2], [3]; simulators such as StarPerf [30] were built to quantify this time-varying behavior. Early work routed around mobility with distributed, delay-oriented algorithms that recompute next hops as the topology evolves [31]. Later classical approaches absorb mobility through topology abstractions; the virtual node model of Lu *et al.* [32] binds logical nodes to earth-fixed footprints so that upper-layer routing observes a quasi-static grid. Optimization-based satellite TE has explored segment routing [33], distributed clustering that optimizes elephant flows separately [34], and supervised learning for distributed split-ratio prediction [35]. Wu *et al.* proposed SaTE [8], the state-of-the-art learning-based TE for satellite constellations: it formulates a heterogeneous graph over satellites, demands, paths, and links, prunes zero-traffic relations to make training tractable on a single GPU, and selects representative topologies via Graph2Vec and determinantal point process (DPP) sampling so that one trained model generalizes across topology snapshots with millisecond inference. In parallel, Yang *et al.* proposed FNC [36], [37], which targets dynamic traffic allocation for earth-observation constellations, replacing RL with end-to-end imitation learning and substituting iterative constraint solvers with normalization-based feasibility layers. Notably, FNC’s imitation learning requires expert allocation labels computed from complete training matrices. Our offline training also uses complete traffic to evaluate rewards, but does not require expert allocation labels; the distinction is not the availability of training-time ground truth. Both SaTE and FNC, like their WAN counterparts, assume the traffic matrix is fully observable at decision time. Unlike SaTE and FNC, SpaTE performs TE from sparse observations, down to a 2% observation ratio (Table 1).

TABLE 1  
Comparison with the related works.

|                    | Teal [6] | SaTE [8] | TEST [14] | FNC [36] | Ours |
|--------------------|----------|----------|-----------|----------|------|
| Satellite scenario | ×        | ✓        | ×         | ✓        | ✓    |
| Dynamic topology   | ×        | ✓        | ×         | ✓        | ✓    |
| Sparse observation | ×        | ×        | ✓         | ×        | ✓    |
| Temporal modeling  | ×        | ×        | ✓         | ×        | ✓    |
| Mask-aware model   | ×        | ×        | ×         | ×        | ✓    |
| Neighbor estimate  | ×        | ×        | ×         | ×        | ✓    |
| Demand-set drift   | ×        | partial  | ×         | ×        | ✓    |

## 2.3 TE with Incomplete Traffic Observations

Network-wide traffic measurement is expensive: collecting a complete traffic matrix requires instrumenting all ingress nodes at fine time granularity, which is particularly onerous on resource-constrained satellite platforms. Even in terrestrial networks, measurement systems trade accuracy for resources by sampling or sketching flow records [38], [39], so operators routinely work with partial counts. A classical workaround is a two-stage pipeline: first estimate or complete the traffic matrix via network tomography (Vardi [10]; Zhang *et al.* [11]), matrix completion [40], or compressive sensing [12], then optimize routing on the estimate. However, reconstruction errors propagate to and degrade the downstream allocation, since a single reconstructed matrix is handed to a separate optimizer. Guo *et al.* proposed TEST [14], which, to our knowledge, is the first *end-to-end* approach to learn routing policies directly from sparsely measured traffic matrices rather than first reconstructing them; it integrates a Transformer over historical sparse sequences with a graph convolutional network (GCN) over the topology, fills unobserved entries with zeros, compensates with synthesized training augmentation, and targets terrestrial networks with static topologies. In the graph learning literature, Huo *et al.* [13] addressed message passing under missing node features via teacher-student distillation, and You *et al.* [41] cast imputation itself as edge prediction on a bipartite graph, but such techniques have not been applied to network TE. No prior work, to our knowledge, performs TE from sparse traffic observations in satellite networks, where the difficulty is compounded by time-varying topologies and drifting demand sets. Unlike TEST, which fills unobserved entries with zeros and targets static terrestrial networks, SpaTE replaces missing entries with learnable mask embeddings, gates their message passing, and handles LEO dynamics—time-varying topologies and drifting demand-pair sets—via demand pruning with size-invariant weights, enabling train-once, zero-retraining deployment; it further uses a source-neighbor estimate for optional congestion-aware refinement without consulting the unobserved ground-truth matrix (Table 1).

## 3 MODEL AND PROBLEM FORMULATION

### 3.1 Network and Traffic Model

We model the constellation at control epoch  $t$  as a directed graph  $G^t = (V, E^t)$ , where  $V$  is the set of  $N$  satellites and  $E^t$  the set of ISLs, each link  $e$  with capacity  $c_e$ . Time is slotted into control epochs; within an epoch the topology snapshot

TABLE 2  
Main Notations.

| Symbol           | Definition                                             |
|------------------|--------------------------------------------------------|
| $G^t = (V, E^t)$ | constellation graph at epoch $t$ ; $c_e$ link capacity |
| $D^t, d_i$       | demand matrix; volume of demand $i$                    |
| $\mathcal{K}^t$  | active demand set at epoch $t$                         |
| $\kappa, P_i$    | candidate paths per demand; path set of demand $i$     |
| $r_i, f_{i,k}$   | split ratios of demand $i$ ; flow on path $p_{i,k}$    |
| $M, \rho$        | binary observation mask; observability ratio           |
| $O^t, H^t$       | sparse observation $D^t \odot M$ ; last- $L$ history   |
| $\phi$           | learnable mask embedding for unmeasured demands        |
| $A, \tilde{A}$   | bipartite adjacency; its mask-gated variant            |
| $H_S, H_T, H_F$  | spatial, temporal, and fused embeddings                |
| $\pi_\theta, R$  | TE policy network; ground-truth reward                 |

is fixed, while across epochs both  $E^t$  and the traffic change as satellites move.

Traffic is described by a demand matrix  $D^t \in \mathbb{R}^{N \times N}$ , where  $d_i$  denotes the volume of the  $i$ -th source-destination pair. Since satellite traffic is geographically concentrated, we maintain an *active demand set*  $\mathcal{K}^t$ , the pairs with nonzero demand in recent history, and only these are modeled. Each demand  $i \in \mathcal{K}^t$  is served by  $\kappa$  pre-selected candidate paths  $P_i = \{p_{i,1}, \dots, p_{i,\kappa}\}$  ( $\kappa = 4$  edge-disjoint shortest paths in our implementation). The TE decision for demand  $i$  is a split-ratio vector  $r_i \in \mathbb{R}^\kappa$ , allocating flow  $f_{i,k} = r_{i,k} \cdot d_i$  to path  $p_{i,k}$ ; the ratios satisfy  $r_{i,k} \geq 0$  and  $\sum_k r_{i,k} = 1$ , so the full offered demand is assigned to candidate paths. This conservation condition does not guarantee that the assigned load fits within link capacities. Main notations are summarized in Table 2.

### 3.2 TE with Complete Observation

With the complete demand matrix available, TE is the classic path-based allocation problem. We focus on load balancing, the standard objective in satellite TE, where hot ISLs over populated regions dictate quality of service [8], [33], [34]. It minimizes the MLU:

$$\min_{\{r_{i,k}\}} \theta \quad (1)$$

$$\text{s.t.} \quad \frac{1}{c_e} \sum_{e \in p_{i,k}} r_{i,k} d_i \leq \theta, \quad \forall e \in E^t, \quad (2)$$

$$\sum_{k=1}^{\kappa} r_{i,k} = 1, \quad \forall i \in \mathcal{K}^t, \quad (3)$$

$$r_{i,k} \geq 0, \quad \forall i, k, \quad (4)$$

where Eq. (3) enforces flow conservation: the full offered demand enters the load calculation, so  $\theta$  cannot be lowered by withholding traffic. There is no constraint  $\theta \leq 1$ . An MLU above one indicates that the assigned load exceeds capacity on at least one link; an optimum  $\theta^* > 1$  means that the offered demands cannot all fit within the available capacities over the selected candidate paths. The metric measures overload rather than delivered throughput. Sustained operation in this regime would require additional capacity, admission control, or traffic shaping, none of which is modeled here. The problem is a linear program solvable to optimality, but at constellation scale its solve time exceeds the control epoch [8]. More fundamentally, the link-load term in Eq. (2)

requires the volume  $d_i$  of every active demand. Sparse-observation TE removes exactly this information from the controller.

### 3.3 TE with Sparse Observation

#### 3.3.1 Observation Model

At each epoch only a subset of demands is measured. A binary mask  $M \in \{0, 1\}^{|\mathcal{K}^t|}$  marks demand  $i$  as measured ( $M_i = 1$ ) or not ( $M_i = 0$ ), and the controller receives the sparse observation

$$O^t = D^t \odot M, \quad \text{together with } M \text{ itself}, \quad (5)$$

where  $\odot$  is the element-wise product. We consider two measurement granularities. Under *flow-level* metering a fraction  $\rho$  of the active demand pairs is sampled, so the hidden entries are scattered across the matrix. Under *node-level* metering a fraction  $\rho$  of satellites meters all of their outgoing demands, which matches deployments where measurement capability resides on a subset of satellites; an unmeasured source then hides an entire demand row at once. Fig. 2 contrasts both against the ground-truth matrix from our Starlink trace: at the same  $\rho$  they hide a comparable number of demands but differ in structure. The controller additionally retains the last  $L$  sparse observations  $H^t = (O^{t-L+1}, \dots, O^t)$ .

#### 3.3.2 Problem Statement

A key observation makes decision-making under unknown demands physically executable: the decision variables are split *ratios*, not absolute volumes. Once installed in the forwarding plane,  $r_i$  is applied multiplicatively to whatever traffic actually arrives, so a decision for demand  $i$  requires no knowledge of  $d_i$ . Sparse-observation TE then seeks a policy  $\pi$  that maps the observable state to split ratios for *all* active demands,

$$\{r_i\}_{i \in \mathcal{K}^t} = \pi(H^t, M, G^t), \quad (6)$$

minimizing the *ground-truth* MLU. Let  $f_{i,k}(\pi) = \tilde{r}_{i,k} \cdot d_i$  denote the offered load assigned to path  $p_{i,k}$ , where  $\{\tilde{r}_i\}$  are the policy's ratios after optional neighbor-assisted rebalancing. The ratios remain conservation-feasible by Eq. (3); the rebalancing only shifts traffic among candidate paths in an attempt to reduce residual overload. The sparse TE objective is

$$\min_{\pi} \mathbb{E}_t \left[ \max_{e \in E^t} \frac{1}{c_e} \sum_{e \in p_{i,k}} f_{i,k}(\pi) \right], \quad (7)$$

where the expectation is over traffic and topology dynamics. The link loads are evaluated with the true demands  $d_i$ : an unmeasured demand still contributes to the offered link load even though the policy could not see it, so misjudging it can inflate realized MLU. The problem is a partially observable sequential decision problem: unlike Eqs. (1) to (4), where the same  $D^t$  appears in both the decision and the constraints, here the decision variables and the evaluation live in different information sets: the policy acts on  $(O, M)$  only, while feasibility and performance are settled by the hidden  $D$ . Intuitively, the policy is graded against an exam it never gets to read in full.

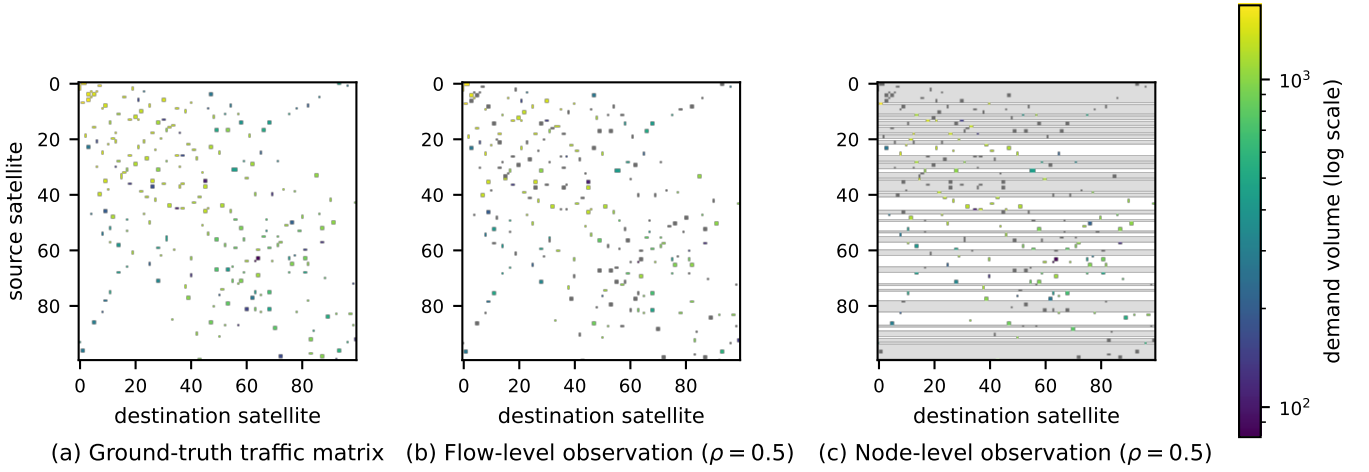


Fig. 2. Traffic and observation are both sparse, and the two metering granularities hide demands differently. **(a)** A ground-truth traffic matrix from the Starlink  $22 \times 72$  trace (100 busiest satellites shown); traffic concentrates on a few source–destination pairs. **(b)** Flow-level metering hides scattered demands (dark gray) and leaves the rest observed (colored). **(c)** Node-level metering hides demands by source: light gray bands are satellites that perform no metering, so their entire demand row is unavailable. At  $\rho = 0.5$  the two panels hide a comparable number of demands, but node-level metering removes them in whole rows, so the controller must allocate demands it has never seen rather than fill in isolated missing entries.

### 3.4 Solution Paradigm

Three solution families exist for Eq. (7). *Solve-with-zeros* plugs  $O$  directly into the LP in Eqs. (1) to (4), implicitly assuming unmeasured demands are zero; the resulting policy leaves no headroom on the links that unmeasured traffic actually traverses, so realized MLU can be far above the optimum. *Two-stage* approaches first estimate  $\hat{D}$  from  $(H, M)$ , via interpolation, tensor completion, or compressive sensing [10], [11], [12], and then solve the LP on  $\hat{D}$ ; estimation is optimized for reconstruction error, which is misaligned with allocation quality, and errors propagate un-damped into the allocation. We instead adopt *end-to-end learning*: parameterize  $\pi_\theta$  as a neural network and optimize  $\theta$  directly against the TE objective,

$$\max_{\theta} \mathbb{E}[R(\pi_\theta(H, M, G), D)], \quad (8)$$

where  $R$  is negative realized MLU from Eq. (7), evaluated on the true  $D$ . Since  $R$  is non-differentiable through the capacity-constrained network and the true  $D$  is available only as a training-time signal, we optimize Eq. (8) with multi-agent reinforcement learning, in which each demand acts as an agent sharing  $\theta$  and receives a counterfactual multi-agent (COMA) [42] style marginal-contribution reward. This paradigm makes the observation model in Eq. (5) a property of the *input*, and the optimization target in Eq. (7) a property of the *training signal*, cleanly separating what the policy can see from what it is optimized for.

## 4 SPATE DESIGN

### 4.1 Overview

SpaTE solves the sparse-observation TE problem of Section 3.3 in four stages, numbered 1–4 in Fig. 3. Given the sparse observation  $(O^t, M)$  and history  $H^t$ , (1) unmeasured demands are represented by a learnable mask embedding (Section 4.2); (2) a mask-gated FlowGNN propagates information between candidate paths and links without letting

#### Algorithm 1 SpaTE inference at control epoch $t$

**Require:** sparse history  $H^t$ , mask  $M$ , topology  $G^t$ , weights  $\theta^*$

**Ensure:** split ratios  $\{r_i\}_{i \in \mathcal{K}^t}$

- 1:  $\mathcal{K}^t \leftarrow$  active demand set from recent history  $\triangleright$  pruning, Section 4.5
- 2: rebuild bipartite graph and gated adjacency  $\tilde{A}$  from  $\mathcal{K}^t, M$   $\triangleright$  Eq. (10)
- 3:  $x_i \leftarrow M_i d_i + (1 - M_i) \phi$  for all  $i$   $\triangleright$  mask embedding, Eq. (9)
- 4:  $H_S \leftarrow \text{GATEDFLOWGNN}(x, \tilde{A})$   $\triangleright$  spatial embeddings
- 5:  $H_T \leftarrow \text{TEMPORALENCODER}(H^t)$   $\triangleright$  temporal embeddings
- 6:  $H_F \leftarrow \text{CROSSATTENTION}(H_T, H_S)$   $\triangleright$  Eq. (11)
- 7:  $\{r_i\} \leftarrow \text{POLICYHEAD}([H_S \parallel H_F])$
- 8:  $\{r_i\} \leftarrow \text{NEIGHBORREFINE}(\{r_i\}, G^t, \hat{D}^t)$   $\triangleright$  optional deployment refinement
- 9: **return**  $\{r_i\}$

placeholders contaminate link states (Section 4.3); (3) a Transformer encodes the observation history and fuses it with spatial embeddings via per-demand cross-attention (Section 4.4); and (4) a shared policy head outputs split ratios per demand. An optional neighbor-assisted pass refines residual overloads using only an observation-derived estimate (Section 4.6). All learnable weights are shared across demands and independent of the demand-set size, which Section 4.5 exploits for zero-retraining deployment under constellation dynamics. The path–link backbone and shared-policy training use Teal [6] as their implementation basis; mask-aware inputs, asymmetric propagation, and temporal fusion adapt the allocator to sparse satellite measurements. Algorithm 1 summarizes one inference epoch.

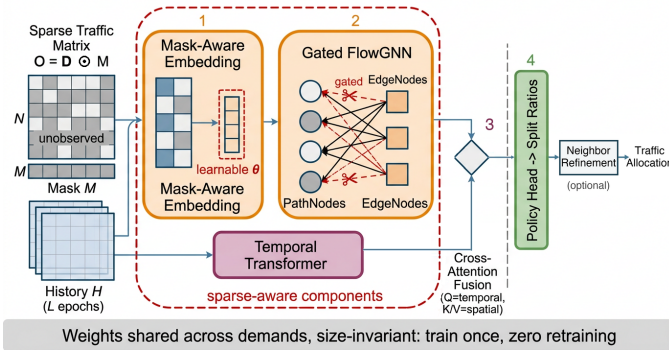


Fig. 3. SpaTE overview. The numbers 1–4 in the figure mark the four stages of Section 4.1: 1 mask-aware demand embedding; 2 mask-gated FlowGNN (Section 4.3); 3 temporal encoding and per-demand cross-attention; and 4 split-ratio prediction. The optional neighbor-assisted refinement uses observation-derived traffic estimates to rebalance paths; it is not ADMM or a projection onto link-capacity constraints.

## 4.2 Mask-Aware Demand Representation

A sparse observation is fundamentally ambiguous: a zero entry may mean “no traffic” or “not measured”. Zero-filling, the de-facto treatment in prior sparse TE [14], collapses the two cases and systematically biases the allocator toward starving unmeasured demands. SpaTE instead assigns each unmeasured demand a *learnable mask embedding*  $\phi \in \mathbb{R}^k$  (one scalar per candidate path):

$$x_i = M_i \cdot d_i + (1 - M_i) \cdot \phi, \quad (9)$$

so the model learns a data-driven prior for missing traffic during end-to-end training, and, critically, the input itself distinguishes “not measured” from “zero”. A learned placeholder token is the same device that masked language models use to mark positions withheld from the input [43]. Learnable imputation has proven effective for incomplete graph features [13], [41]; we bring it to TE, where the imputed value directly steers resource allocation. The embedding is trained jointly with the policy under the TE objective in Eq. (8) rather than a reconstruction loss, so  $\phi$  converges to values that maximize allocation quality, not reconstruction accuracy.

## 4.3 Mask-Gated FlowGNN

The spatial encoder, labelled Gated FlowGNN in Fig. 3, represents competition for ISL capacity through a path-link bipartite graph. Each candidate path is a PathNode connected to the EdgeNodes for the ISLs it traverses. This representation lets demands on different satellite pairs exchange information through shared links along multi-hop routes. Each layer combines GCN-style normalized graph aggregation [44] with a shared transformation over the candidate paths of each demand; skip connections retain the original inputs. After  $\Lambda$  layers, the PathNode features form the spatial embeddings  $H_S$  used in Section 4.4.

The sparse-observation modification is a mask-controlled direction of propagation. Without it, placeholders for unmeasured satellite demands enter shared link embeddings and can affect other demands using the same ISLs, consistent with feature-structure interference in incomplete

graphs [13]. We gate path-to-link messages while preserving the reverse direction. Let  $A$  be the normalized bipartite adjacency; the mask-gated operator is

$$\tilde{A}[e, p] = M_p \cdot A[e, p], \quad \tilde{A}[p, e] = A[p, e], \quad (10)$$

where  $M_p$  is the demand-level mask lifted to path  $p$ . Unmeasured demands thus remain *listeners*: they sense link states, which are shaped by measured traffic, but inject no fabricated volume into them. The gate is a fixed binary function of the mask with no extra learnable parameters; its weights are prepared when the graph is built. It plays the role that learned attention coefficients play in attention-based GNNs [45], except that here the weighting is dictated by measurement status rather than learned, which is what makes the following guarantee exact rather than empirical. The asymmetric gate yields a formal isolation property:

**Proposition 1 (Placeholder isolation).** Under the gated propagation in Eq. (10), the embeddings of all EdgeNodes and of all PathNodes belonging to measured demands are independent of the mask embedding  $\phi$ ; for fixed network weights,  $\phi$  influences only the policy-head outputs of unmeasured demands, before optional refinement.

*Proof:* Consider first the spatial stream, by induction on GNN layers. At layer 0, only unmeasured PathNodes carry  $\phi$ . At layer  $l$ , an EdgeNode aggregates path-to-edge messages with weights  $\tilde{A}[e, p] = M_p A[e, p]$ , which vanish for every unmeasured path; hence EdgeNode embeddings depend only on measured-path inputs and their own history, none of which contains  $\phi$ . A measured PathNode aggregates EdgeNode embeddings ( $\phi$ -free by the above) and its own  $\phi$ -free input; the per-demand layers mix only paths of the same demand, which share the same mask value. The temporal and fusion streams preserve the property: the temporal encoder acts per path on the raw observation history  $H^t$ , which contains no  $\phi$  by construction, and the cross-attention of Eq. (11) operates strictly within each demand, so the fused features of a measured demand combine only its own  $\phi$ -free temporal and spatial embeddings. Thus  $\phi$  never enters any  $\phi$ -free branch, and only unmeasured PathNodes, and consequently their own split ratios, depend on  $\phi$ .  $\square$

This property limits direct propagation of placeholder errors through the learned allocator; it does not guarantee lower MLU. It also does not extend to the optional refinement, where all demands interact through estimated link loads.

## 4.4 Temporal-Spatial Fusion

Satellite traffic exhibits strong short-term temporal correlation; the recent history of sparse observations therefore carries recoverable information about currently missing entries. SpaTE encodes, for each candidate path, its observation sequence over the last  $L$  epochs with a lightweight Transformer [15]: scalar volumes are log-compressed, which we find essential for training stability, and projected to  $d_{\text{model}}$  dimensions, summed with learnable positional embeddings, and passed through a Transformer encoder; the representation at the latest step forms the temporal embedding  $H_T$ . Rather than concatenating temporal and spatial features

naively, we fuse them with *per-demand cross-attention*: within each demand, temporal embeddings of its  $\kappa$  candidate paths act as queries attending to the  $\kappa$  spatial (GNN) embeddings  $H_S$  as keys and values,

$$H_F = \text{softmax}\left(\frac{W_q H_T (W_k H_S)^\top}{\sqrt{d}}\right) W_v H_S, \quad (11)$$

and the fused features  $[H_S \parallel H_F]$  feed the policy head. Intuitively, the temporal stream decides *which* spatial evidence to trust for each path: when the current entry is missing, attention shifts toward paths and links whose states corroborate the historical pattern. The full pipeline infers in under 0.4 s per snapshot on the 1,584-satellite constellation, well within the 5-minute control-loop period.

## 4.5 Scaling to Dynamic Constellations

### 4.5.1 Demand Pruning

Modeling all  $N(N-1)$  pairs of a 1,584-satellite constellation yields 2.5M demands and tens of millions of path variables, which is infeasible for online control. Satellite traffic, however, is geographically concentrated: only 0.25% of pairs ever carry traffic in our dataset. SpaTE instantiates PathNodes only for the demand-pair union observed in historical traffic matrices, shrinking the graph by  $\sim 400\times$  while retaining all nonzero demands in that historical union, in the same spirit as SaTE's traffic pruning [8].

### 4.5.2 Zero-Retraining under Drift

Pruned demand sets are not static: as satellites move, the active pair set drifts (we measure  $\sim 10\%$  turnover across our trace). Rather than retraining, SpaTE exploits a structural property of its architecture: every learnable component (GNN layers, mask embedding, temporal encoder, fusion and policy heads) is shared across demands and independent of the demand-set size. The demand set, candidate paths, and the bipartite graph are runtime *inputs*, not parameters. At each control epoch the controller rebuilds the graph from the current active set (a millisecond-scale index operation on +Grid constellations, where candidate paths depend only on the orbital displacement between endpoints and reduce to  $O(N)$  path templates) and reuses the trained weights as-is. Demands outside the modeled set, necessarily sporadic mice flows, fall back to shortest-path routing until the next set refresh. This reuse is justified by the following structural property:

**Proposition 2 (Size invariance and permutation equivariance).** The parameter set of SpaTE is independent of  $|\mathcal{K}^t|$ ,  $N$ , and  $|E^t|$ , and the mapping from inputs to split ratios is equivariant to any permutation of the demand indices.

*Proof:* Every learnable component is a per-node or per-demand operator with shared weights: the mask embedding  $\phi$  and temporal encoder act on each path token, the GNN layers combine node-wise linear maps with sparse aggregation, the fusion and policy heads act within each demand. None of their parameter shapes involves  $|\mathcal{K}^t|$ ,  $N$ , or  $|E^t|$ . Permuting demand indices permutes the rows of every input and of the bipartite adjacency consistently; since shared node-wise operators commute with permutations

and sparse aggregation is index-based, all intermediate embeddings, and hence the outputs, are permuted identically.  $\square$

The proposition places SpaTE in the same family as inductive graph encoders that share node-wise weights to generalize to graphs unseen in training [46] and as permutation-invariant set architectures [47]. The distinction is what varies at deployment: not only the graph, but the demand set itself, which drifts as footprints move.

## 4.6 Training and Neighbor-Assisted Refinement

The mask embedding, spatial encoder, temporal encoder, and fusion layers are trained jointly against the realized MLU objective in Eq. (8). Each demand acts as an agent sharing the policy network. The policy outputs a Gaussian mean per path, and actions are sampled during training. A COMA-style counterfactual reward [42] assigns credit to each demand by estimating its marginal contribution to the global objective: for demand  $i$ ,

$$A_i = \mathbb{E}_{\tilde{r}_i \sim \pi'} [R(r_{-i}, r_i) - R(r_{-i}, \tilde{r}_i)], \quad (12)$$

where  $r_{-i}$  denotes the allocations of all other demands and  $\tilde{r}_i$  is a counterfactual action resampled from a reference distribution  $\pi'$ ; the policy gradient weights each agent's log probability by  $A_i$ . Algorithm 2 summarizes offline training. The policy receives only sparse inputs, whereas rewards are evaluated using complete simulated or historical demands. Unlike imitation learning [36], this procedure does not require optimal allocation labels for each training snapshot. It does, however, require access to those complete demands during offline training; the sparse-observation restriction applies to policy inputs and deployment-time decisions.

**Neighbor-assisted refinement.** The policy head uses a softmax, so its split ratios satisfy flow conservation in Eq. (3) by construction. A short post-processing pass may nevertheless improve MLU by moving a fraction of each demand from its most congested candidate path to its least congested one. This is a conservation-preserving rebalancing heuristic, not ADMM or a capacity-feasibility projection. Each pass preserves the per-demand total but does not guarantee that every link's load falls below capacity. Our deployed setting uses two rounds, each containing ten lightweight sweeps.

The key difficulty under sparse observation is not the rebalancing rule but how to estimate the link loads used to rank paths. A controller that meters only a fraction  $\rho$  of demands cannot compute true utilization. Consequently, a refinement implementation that reads the ground-truth matrix (denoted ORACLE) is unsuitable for deployment. SpaTE computes  $\hat{D}^t$  from the observation instead: every unobserved demand is assigned a *neighbor estimate* from observed demands sharing its source satellite, scaled by the observed per-source traffic. This choice exploits geographic locality in satellite traffic and is used only to estimate path congestion; the learned allocator remains the main decision module. The evaluation compares this estimate with zero filling and the undeployable oracle to isolate the value of the traffic estimate rather than presenting post-processing as the central contribution.

**Algorithm 2** Offline training of SpaTE

**Require:** historical trace  $\{(D^t, G^t)\}$ , mask  $M$ , learning rate  $\eta$

**Ensure:** trained weights  $\theta^*$  (incl. mask embedding  $\phi$ )

- 1: build demand set  $\mathcal{K}$  and graph from training-slice TMs
- 2: **for** each epoch **do**
- 3:   **for** each snapshot  $t$  in the training slice **do**
- 4:     form sparse input  $(H^t, M)$ ; sample actions  $r \sim \pi_\theta(\cdot \mid H^t, M, G^t)$
- 5:     compute counterfactual rewards  $\{A_i\}$  via Eq. (12) **using the true**  $D^t$
- 6:      $\theta \leftarrow \theta + \eta \sum_i A_i \nabla_\theta \log \pi_\theta(r_i)$
- 7:   **end for**
- 8: **end for**
- 9: **return**  $\theta^*$

**4.7 Complexity Analysis**

Let  $D = |\mathcal{K}^t|$  be the number of active demands,  $\kappa$  the candidate paths per demand,  $E$  the number of ISLs, and  $\bar{\ell}$  the average path length. The gated FlowGNN performs message passing over the bipartite graph whose edge count is  $O(D\kappa\bar{\ell})$ ; with  $\Lambda$  layers its cost is  $O(\Lambda D\kappa\bar{\ell}d)$  for hidden width  $d$ . The temporal encoder applies a Transformer of length  $L$  independently to each of the  $D\kappa$  path tokens, costing  $O(D\kappa L^2 d)$ , and the per-demand cross-attention over  $\kappa$  paths adds  $O(D\kappa^2 d)$ . Since  $\kappa$ ,  $L$ ,  $\bar{\ell}$ , and  $d$  are small constants fixed by design, the end-to-end inference cost is *linear* in the number of active demands,  $O(D)$ , and all operations are batched across demands on the GPU. Crucially, the parameter count is independent of  $D$ ,  $N$ , and  $E$ : the same weights apply to any demand set, which is what enables zero-retraining reuse (Section 4.5). In our configuration the entire model has merely 4,468 trainable parameters, small enough to be replicated on resource-constrained satellite platforms. Demand pruning reduces  $D$  from  $N(N-1)$  to the observed support (roughly  $400\times$  on our constellation), turning an otherwise intractable problem into one solved well within the control period.

**5 EVALUATION****5.1 Setup**

**Constellation.** Our main testbed is a Starlink Shell-1 Walker constellation [48] interconnected in a +Grid pattern (Table 3); orbital dynamics are derived from the Satellite Tool Kit (STK). With 1,584 satellites, an average path length of 23.51 hops, and a diameter of 47, capacity conflicts between demands are far more frequent than in the terrestrial WANS used in prior TE studies [6]. For the cross-scale check (Section 5.2.2) we use a one-third-scale instance of the same architecture (24 planes  $\times$  22 satellites, 528 nodes).

**Traffic.** Traffic matrices are generated by random city sampling [49]: cities are drawn with probability proportional to their population, each sampled city pair is attached to its nearest satellites, and per-pair flows are aggregated into satellite-level demands (Table 4). We use 101 consecutive snapshots (one every 5 minutes), with snapshots 0–79 for training, 80–89 for validation, and 90–100 for testing, a strict temporal split with no leakage. Under the chosen

TABLE 3  
Simulation parameters of the Starlink constellation.

| Parameter                   | Value      |
|-----------------------------|------------|
| Height of LEO satellites    | 550 km     |
| Constellation configuration | Walker     |
| Total number of satellites  | 1,584      |
| Number of orbital planes    | 72         |
| Satellites per plane        | 22         |
| Inclination of orbits       | $55^\circ$ |
| Phase factor                | 3          |
| Inter-satellite links       | 6,336      |

TABLE 4  
Statistics of the population-based traffic trace (Mbps).

| Metric                         | Value             |
|--------------------------------|-------------------|
| Snapshots (5-min interval)     | 101               |
| Nonzero demand, max            | 1,800             |
| Nonzero demand, mean $\pm$ std | $159.5 \pm 242.3$ |
| Total demand per snapshot      | 890,242           |
| Active pairs per snapshot      | 5,583             |
| Active-pair union (trace)      | 6,334             |

ISL capacities, a path-restricted LP with complete traffic information attains a mean MLU of  $\approx 1.38$  on the test snapshots. The main setting therefore tests load balancing under overload, not guaranteed capacity-feasible delivery; Section 3.2 defines the meaning of MLU above one. Demand pruning (Section 4.5) restricts the modeled demand set to the  $\sim 6.3K$  pairs that ever carry traffic, out of 2.5M node pairs.

**Metric and protocol.** Our metric is the *realized MLU* evaluated against ground-truth demands (lower is better); flow conservation in Eq. (3) is enforced by construction, so MLU cannot be gamed by under-serving traffic. We sweep the observation ratio  $\rho \in \{0.02, 0.05, 0.1, 0.3, 0.5\}$  and use five independent experimental repeats per configuration, controlled by random seeds 0–4. Matching repeat indices use the same seed across methods. Unless stated otherwise, every method is paired with the *identical deployable post-processing layer*: two rounds of ten rebalancing sweeps, whose utilization estimate reads only the sparse observation with neighbor fill (OBS-NBR, Section 4.6), so that no method receives unavailable demand information and the allocator remains the only factor that varies. The ORACLE variant, which reads the ground-truth matrix, appears only in the neighbor-estimation diagnostic. Each plotted point is the mean MLU over the same eleven chronological test snapshots for one experimental repeat. We show all five repeats and summarize them by mean $\pm$ standard deviation, median, and range. Because the snapshots in a trace are temporally correlated, headline comparisons are paired on  $(\rho, \text{repeat})$ , not on snapshots, and are assessed with two-sided sign tests. To isolate the allocator from this shared layer, we additionally rerun the full  $6 \times 5 \times 5$  matrix with refinement disabled. Separately, we pair source-row (node-level) metering with the same flow-level deployable cells for three methods, three observation ratios, and seeds 0–2. These two diagnostics are reported separately from the deployed main comparison.

**Controlled comparisons.** The learned variants below

TABLE 5

Model and training hyperparameters (defaults held fixed across all  $\rho$  and repeats).

| Parameter                                    | Value                        |
|----------------------------------------------|------------------------------|
| FlowGNN layers $\Lambda$                     | 6                            |
| Candidate paths $\kappa$                     | 4                            |
| Temporal $d_{\text{model}}$ / heads / layers | 16 / 2 / 1                   |
| Fusion dim $d_{\text{fuse}}$                 | 8                            |
| History length $L$                           | 3                            |
| COMA samples per update                      | 30                           |
| Mask-embedding init                          | 57.0 (median nonzero demand) |
| Learning rate                                | $1 \times 10^{-4}$           |
| Epochs (warmup / early stop)                 | 60 (4 / on)                  |
| Restart screening                            | 3 inits                      |
| Batch size                                   | 20                           |
| Untrained control                            | 0 epochs, 1 init             |
| Neighbor refinement (deployed)               | 2 rounds $\times$ 10 sweeps  |
| Trainable parameters                         | 4,468                        |

share the training pipeline, path sets, and optional neighbor-assisted refinement of SpaTE. They isolate sparsity treatments within this implementation; they are not independent reproductions of the cited systems:

- *zero-fill*: the sparse adaptation of full-observation systems such as Teal [6] and SaTE [8], which cannot run on an incomplete matrix; unobserved entries are set to zero and the model is trained end to end on the filled input. SaTE’s heterogeneous-graph allocator targets the same MLU objective with the same pruning, so this variant is its closest sparse-input proxy.
- *mean-interp*: a two-stage complete-then-optimize baseline that fills unobserved entries with the mean of observed demands before allocation.
- *nbr-fill*: a stronger two-stage baseline that fills each unobserved demand from observed demands sharing its source satellite, the same estimate used by the optional refinement.
- *TEST-inspired*: a controlled baseline motivated by TEST [14], with zero-filled input, a Transformer over the observation history ( $L = 3$ ), and an ungated GCN. We use the same paths, objective, and training pipeline as the other methods rather than claiming reproduction of TEST’s original terrestrial setup.
- *untrained*: the SpaTE architecture with weights frozen at random initialization, isolating the contribution of training. It is *not* the LP optimum: the LP (Eq. (1)) needs the complete matrix and serves only as a full-information lower-bound reference for MLU ( $\approx 1.38$ ), not as an online sparse-input baseline.

**Implementation.** Table 5 lists the hyperparameters, drawn from the code defaults and held fixed across all  $\rho$  and repeats; all runs use Adam [50]. The untrained control shares the trained model’s random initialization and differs only in skipping gradient updates ( $\text{epochs}=0$ ); it is therefore a clean ablation of the *training* factor with identical architecture, mask embedding, and post-processing.

TABLE 6

Cross-scale check on the 528-satellite instance without post-processing (median MLU and paired training gain over three repeats).

| $\rho$ | Trained | Untrained | Paired gain | SpaTE lower |
|--------|---------|-----------|-------------|-------------|
| 0.3    | 3.69    | 3.83      | −3.2%       | 1/3         |
| 0.05   | 3.04    | 4.02      | +24.4%      | 3/3         |

## 5.2 Performance of SpaTE

Across observation ratios from 2% to 50%, SpaTE achieves a mean realized MLU in the 1.64–1.89 range (the SpaTE bars of Fig. 5), tightening as more demands become visible: from 1.885 at  $\rho = 0.05$  down to 1.639 at  $\rho = 0.5$ . To calibrate these numbers we solve a path-restricted LP that sees the *complete* matrix on the same test snapshots and candidate paths; it attains a full-information optimum of  $\text{MLU} \approx 1.38$ . Observing only a fraction of demands, SpaTE stays within  $1.19$ – $1.37\times$  of this optimum. An MLU of 1.7 means that the assigned load on the busiest link is 70% above its capacity. Flow conservation is maintained, but capacity feasibility is not: the metric does not imply that all offered traffic is delivered within the epoch. These overloaded-load-balancing results cannot establish delay, loss, or admission-control guarantees. The full-observation LP provides a lower bound on achievable MLU for the chosen paths, not an online sparse-input baseline; it is distinct from the *untrained* control, which uses random weights.

### 5.2.1 Inference latency

On the 1,584-satellite constellation, one complete allocation pass (demand-set assembly, graph rebuild, gated GNN, temporal encoder, fusion, policy head, and optional neighbor-assisted refinement) completes in under 0.4 s per snapshot on a single GPU, against a 5-minute control period. Combined with the  $O(D)$  complexity of Section 4.7 and the 4,468-parameter model size, SpaTE fits the compute envelope of a constellation controller.

### 5.2.2 Cross-scale generality

To test whether the allocator depends on constellation scale, we reuse its size-invariant architecture on a one-third-scale Shell-1 instance with 528 satellites and 2,112 ISLs. Table 6 isolates the learned allocation prior without post-processing at two observation ratios. At  $\rho = 0.05$ , training lowers median MLU from 4.02 to 3.04 and improves all three matched repeats. The result at  $\rho = 0.3$  is mixed, indicating that the most informative observation regime shifts with traffic density. The model requires no architectural change as the graph and demand-set sizes change, which supports reuse across constellation scales while avoiding a claim of uniform gains.

This check covers two sizes of a Starlink-like constellation under the population-based traffic model of Table 4. It does not establish generality across different constellation systems, orbital configurations, or traffic-generation methods. Those factors require separately generated and evaluated scenarios; changing constellation size alone does not test them.

TABLE 7

Paired comparison of SpaTE against each controlled variant over 25 ( $\rho$ , repeat) pairs. The middle column counts the pairs in which SpaTE attains the lower MLU (out of 25); there are no ties.  $p$  is a two-sided sign test.

| Control       | SpaTE lower / 25 | $p$    |
|---------------|------------------|--------|
| TEST-inspired | 11               | 0.6900 |
| zero-fill     | 16               | 0.2295 |
| mean-interp   | 16               | 0.2295 |
| nbr-fill      | 15               | 0.4244 |
| untrained     | 14               | 0.6900 |

TABLE 8

Allocator-only comparison with refinement disabled, aggregated over the same 25 ( $\rho$ , repeat) pairs. The middle column counts pairs where SpaTE has lower MLU. Values are unadjusted two-sided sign tests and are reported descriptively across the five ratios.

| Control       | SpaTE lower / 25 | $p$    |
|---------------|------------------|--------|
| TEST-inspired | 7                | 0.0433 |
| zero-fill     | 8                | 0.1078 |
| mean-interp   | 7                | 0.0433 |
| nbr-fill      | 9                | 0.2295 |
| untrained     | 16               | 0.2295 |

### 5.3 Comparison with Sparsity Treatments

We now compare SpaTE against the controlled sparsity-treatment variants under the identical deployable protocol. Fig. 5 summarizes each method as a mean with its standard deviation, and Fig. 4 shows the underlying 150 individual repeats. The mean rankings vary across observation ratios. The error bars describe repeat-to-repeat dispersion; the statistical comparisons below instead use matched repeats. In particular, SpaTE attains a lower MLU than zero filling on 16 of the 25 matched ( $\rho$ , repeat) pairs, but the paired sign test gives  $p = 0.2295$ ; its comparisons with TEST-inspired, mean-interp, nbr-fill, and untrained controls are likewise non-significant (Table 7). Hence the observed rankings must be interpreted together with their variability; they do not establish overall superiority.

The shared neighbor-assisted refinement can redirect traffic away from overloaded paths regardless of which allocator produced the initial split. We therefore reran all six variants without post-processing: the resulting 150 cells give an allocator-only diagnostic. Under this protocol, SpaTE has the lower MLU in 16 of 25 pairs against the untrained control ( $p = 0.2295$ ), but only 7 of 25 pairs against each of TEST-inspired and mean-interp (unadjusted  $p = 0.0433$ ; Table 8). The no-refinement ordering thus does not support a uniform allocator advantage; it also differs from the deployed ordering. This controlled contrast shows that the refinement layer is material to the end-to-end outcome, but it does not attribute a separate causal contribution to each learned component. Initialization variability is another contributor to the mixed rankings. We therefore distinguish end-to-end results under refinement from allocator-only component diagnostics (Section 5.4), rather than treating either as evidence of uniform superiority.

TABLE 9

Paired node-level minus flow-level MLU under the identical deployable protocol (three seeds per cell). Positive values indicate higher MLU under node-level metering.

| Method    | $\rho = 0.1$ | $\rho = 0.3$ | $\rho = 0.5$ |
|-----------|--------------|--------------|--------------|
| SpaTE     | +0.001       | +0.051       | +0.107       |
| zero-fill | -0.116       | -0.031       | -0.105       |
| untrained | -0.068       | +0.147       | -0.016       |

## 5.4 Ablations

### 5.4.1 Effect of training

We isolate the contribution of training by comparing SpaTE with its untrained counterpart under the common neighbor-assisted refinement. Fig. 6 plots the five test averages for each condition, one per experimental repeat. Training attains the lower MLU in 14 of the 25 matched ( $\rho$ , repeat) pairs and the higher in 11, with no ties (two-sided sign test  $p = 0.690$ ). At  $\rho = 0.5$ , for example, the mean is  $1.639 \pm 0.074$  after training versus  $1.757 \pm 0.107$  without training; at the other ratios the paired outcomes are mixed. Thus these data do not establish a statistically significant end-to-end training advantage, but they expose the initialization sensitivity that a single aggregate score would hide.

### 5.4.2 Metering granularity

We next compare the deployed protocol under flow-level metering and node-level metering, where observing a source reveals its entire outgoing demand row. Each entry in Table 9 is a mean difference over the same three seeds; positive values mean that node-level metering gives higher (worse) MLU. The two masks expose nearly the same fractions of eligible pairs, while their observed test-volume fractions are 10.6%, 30.0%, and 49.8% for node-level metering at  $\rho = 0.1$ , 0.3, and 0.5 (respectively 11.6%, 30.6%, and 51.1% for flow-level metering). The paired differences change sign across methods and ratios. Hence this three-seed diagnostic neither establishes that source-row metering is always harder nor that SpaTE has a node-level advantage.

### 5.4.3 Representation components

We next test the three representation choices without post-processing at the two sparsest ratios, where their effect is least likely to be hidden by rebalancing. Fig. 7 shows each experimental repeat. The outcomes are mixed: no configuration dominates the others at both ratios, and the three-repeat diagnostic is insufficient for a component-level performance claim. We therefore retain the placeholder-isolation result of proposition 1 as a structural property, rather than interpreting it as proof that gating, masking, or temporal encoding always lowers MLU.

### 5.4.4 Sensitivity to the neighbor estimate

The optional refinement pass needs an estimated traffic matrix only to rank candidate paths by congestion. Fig. 8 fixes the untrained control at  $\rho = 0.3$  and shows all three experimental repeats. Moving from no post-processing (median 2.833) to an observation with zeros elsewhere (2.295), and then to the source-neighbor estimate (1.740), substantially changes the congestion signal available to the rebalancer.

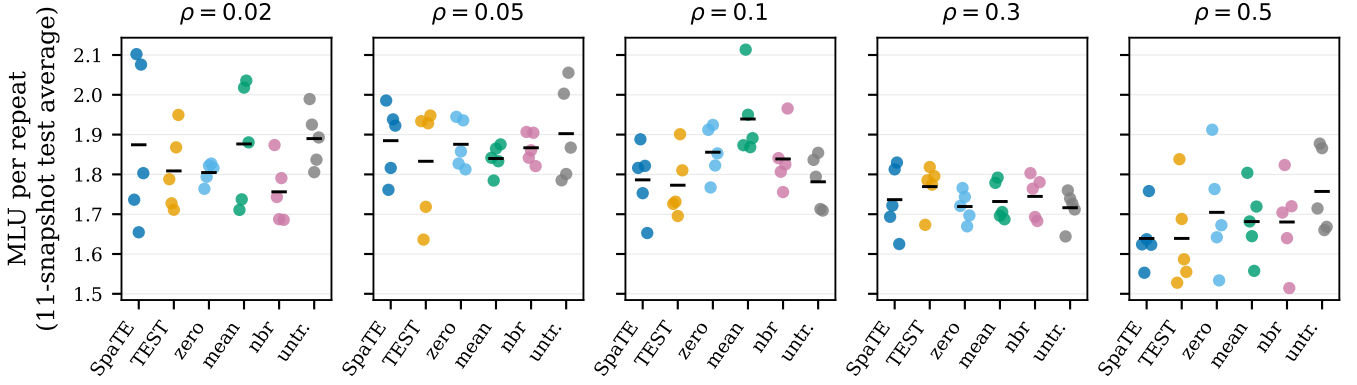


Fig. 4. Controlled sparsity-treatment comparison across five experimental repeats under the same deployed neighbor-assisted refinement. Each marker is one repeat's average MLU over the 11 test snapshots; black bars mark means. The dispersion explains why column-wise rankings based only on a median are not reliable evidence of a general advantage.

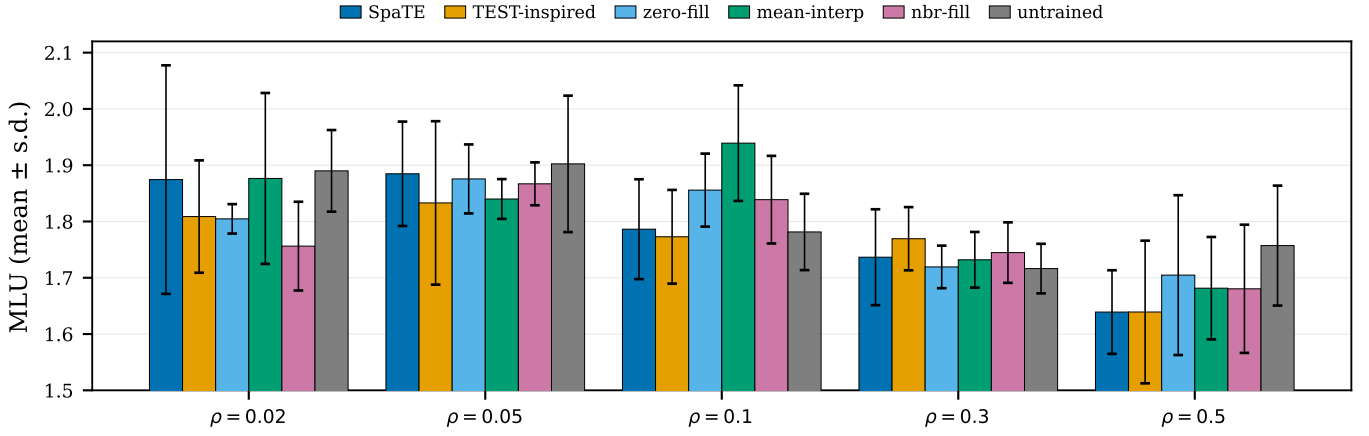


Fig. 5. Controlled sparsity-treatment comparison under the aligned deployable protocol. Bars are means over five experimental repeats and whiskers are one standard deviation. The vertical axis is truncated at 1.5; bar heights above the baseline must not be interpreted as proportional improvements. Lower MLU is better. Statistical comparisons use paired repeats (Table 7), not error-bar overlap.

The oracle reference reaches 1.599, but uses unavailable ground-truth traffic and five rounds of ten sweeps; it is a full-information diagnostic rather than a protocol-matched baseline. The deployable conclusion is therefore limited to the value of the neighbor estimate over zero filling, not a claim that post-processing is the central learned mechanism.

## 6 CONCLUSION

This paper studied MLU-oriented traffic engineering for LEO satellite constellations when telemetry constraints expose only a small fraction of demands. We presented SpaTE, a sparse-aware learned allocator that represents missing measurements explicitly, prevents placeholder traffic from contaminating graph messages, and combines spatial and temporal evidence before predicting path split ratios. Shared parameters and demand pruning keep the architecture independent of the instantaneous demand-set size.

Repeated evaluations on a 1,584-satellite Starlink constellation show that MLU rankings vary with both the observation ratio and model initialization. SpaTE attains

the lower MLU on more matched trials than the zero-fill, mean-interp, and nbr-fill controls, yet none of its external pairwise comparisons is statistically significant. The component diagnostic is also mixed at the two sparsest ratios. These negative results matter: they show that the structural plausibility of a sparse-aware representation does not by itself establish a uniform end-to-end MLU gain, and that comparisons must account for variation across experimental repeats rather than rely on a single summary statistic.

For deployment, a source-neighbor estimate can provide optional post-processing with a useful congestion signal without access to the complete traffic matrix. It is a supporting mechanism, not a substitute for the sparse-aware allocator. Future work should improve robustness across initializations and observation masks through adaptive measurement scheduling, observation-aware model selection, and objectives that explicitly control worst-case MLU.

## REFERENCES

- [1] M. Handley, "Delay is not an option: Low latency routing in space," in *Proceedings of the 17th ACM Workshop on Hot Topics in*

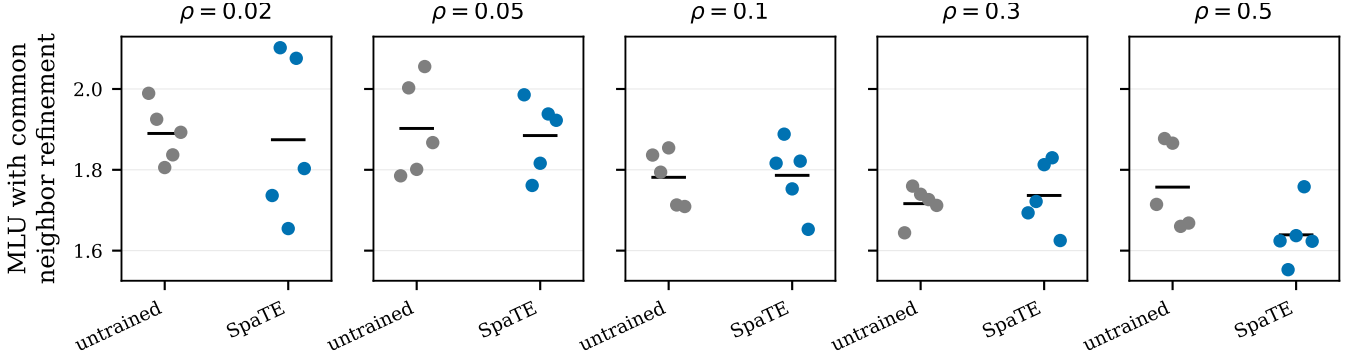


Fig. 6. Effect of training under identical neighbor-assisted refinement. Gray dots denote the untrained control and blue dots denote SpaTE. Each dot is one repeat's mean MLU over the same 11 test snapshots; slight horizontal offsets distinguish the five repeats, and black bars mark their means. Matching repeats share a random seed; no connecting lines are drawn.

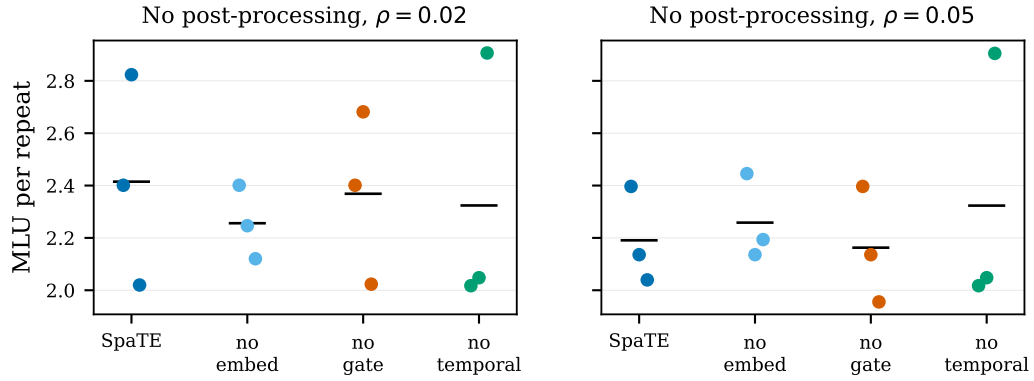


Fig. 7. Low-observability component diagnostic without post-processing (three experimental repeats). Dots are test averages from individual repeats and black bars are means. The mixed ordering supports treating the module choices as design alternatives rather than assuming a uniform component gain.

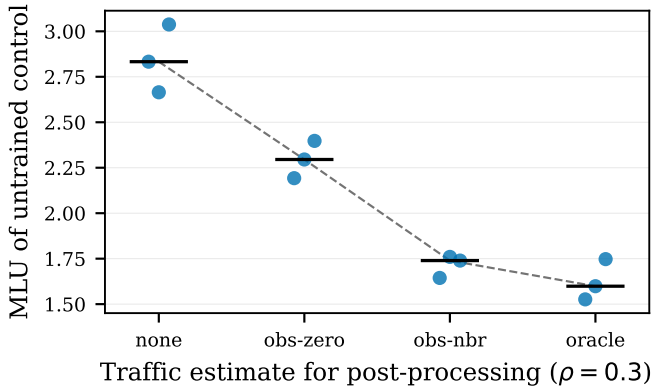


Fig. 8. Sensitivity to the traffic estimate used by post-processing at  $\rho = 0.3$ . Each dot is one experimental repeat; black bars denote medians. The oracle uses unavailable complete traffic and five rounds of ten sweeps; deployed methods use an observation-derived estimate.

*Networks (HotNets)*, 2018.

- [2] D. Bhattacharjee and A. Singla, "Network topology design at 27,000 km/hour," in *Proceedings of the 15th International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*,

- 2019.
- [3] S. Kassing, D. Bhattacharjee, A. B. Águas, J. E. Saethre, and A. Singla, "Exploring the "internet from space" with hypatia," in *Proceedings of the ACM Internet Measurement Conference (IMC)*, 2020.
- [4] S. Jain, A. Kumar, S. Mandal, J. Ong, L. Poutievski, A. Singh, S. Venkata, J. Wanderer, J. Zhou, M. Zhu, J. Zolla, U. Hölzle, S. Stuart, and A. Vahdat, "B4: Experience with a globally-deployed software defined WAN," in *Proceedings of the ACM SIGCOMM 2013 Conference*, 2013, pp. 3–14.
- [5] C.-Y. Hong, S. Mandal, M. Al-Fares, M. Zhu, R. Alimi, K. N. Bollineni, C. Bhagat, S. Jain, J. Kaimal, S. Liang, K. Mendelev, S. Padgett, F. Rabe, S. Ray, M. Tewari, M. Tierney, M. Zahn, J. Zolla, J. Ong, and A. Vahdat, "B4 and after: Managing hierarchy, partitioning, and asymmetry for availability and scale in google's software-defined WAN," in *Proceedings of the ACM SIGCOMM 2018 Conference*, 2018, pp. 74–87.
- [6] Z. Xu, F. Y. Yan, R. Singh, J. T. Chiu, A. M. Rush, and M. Yu, "Teal: Learning-accelerated optimization of WAN traffic engineering," in *Proceedings of the ACM SIGCOMM 2023 Conference*, 2023, pp. 378–393.
- [7] Y. Perry, F. V. Frujeri, C. Hoch, S. Kandula, I. Menache, M. Schapira, and A. Tamar, "DOTE: Rethinking (predictive) WAN traffic engineering," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 1557–1581.
- [8] H. Wu, Y. Han, M. Rajpal, Q. Zhang, and J. Wang, "SaTE: Low-latency traffic engineering for satellite networks," in *Proceedings of the ACM SIGCOMM 2025 Conference*, 2025, pp. 896–916.
- [9] F. Michel, M. Trevisan, D. Giordano, and O. Bonaventure, "A first

- look at starlink performance," in *Proceedings of the ACM Internet Measurement Conference (IMC)*, 2022.
- [10] Y. Vardi, "Network tomography: Estimating source-destination traffic intensities from link data," *Journal of the American Statistical Association*, vol. 91, no. 433, 1996.
  - [11] Y. Zhang, M. Roughan, N. Duffield, and A. Greenberg, "Fast accurate computation of large-scale IP traffic matrices from link loads," in *Proceedings of ACM SIGMETRICS 2003*, 2003.
  - [12] Y. Zhang, M. Roughan, W. Willinger, and L. Qiu, "Spatio-temporal compressive sensing and internet traffic matrices," in *Proceedings of the ACM SIGCOMM 2009 Conference*, 2009.
  - [13] C. Huo, D. Jin, Y. Li, D. He, Y.-B. Yang, and L. Wu, "T2-GNN: Graph neural networks for graphs with incomplete features and structure via teacher-student distillation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.
  - [14] Y. Guo, Z. Huang, M. Ding, and H. Luo, "An end-to-end learning approach for traffic engineering with sparse traffic measurements," *IEEE Transactions on Mobile Computing*, vol. 25, no. 4, pp. 5448–5463, 2026.
  - [15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
  - [16] B. Fortz and M. Thorup, "Internet traffic engineering by optimizing OSPF weights," in *Proceedings of IEEE INFOCOM 2000*, 2000.
  - [17] H. Wang, H. Xie, L. Qiu, Y. R. Yang, Y. Zhang, and A. Greenberg, "COPE: Traffic engineering in dynamic networks," in *Proceedings of the ACM SIGCOMM 2006 Conference*, 2006.
  - [18] C.-Y. Hong, S. Kandula, R. Mahajan, M. Zhang, V. Gill, M. Nanduri, and R. Wattenhofer, "Achieving high utilization with software-driven WAN," in *Proceedings of the ACM SIGCOMM 2013 Conference*, 2013.
  - [19] P. Kumar, Y. Yuan, C. Yu, N. Foster, R. Kleinberg, P. Lapukhov, C. L. Lim, and R. Soule, "Semi-oblivious traffic engineering: The road not taken," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, 2018, pp. 157–170.
  - [20] F. Abuzaied, S. Kandula, B. Arzani, I. Menache, M. Zaharia, and P. Bailis, "Contracting wide-area network topologies to solve flow problems quickly," in *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, 2021, pp. 175–200.
  - [21] D. Narayanan, F. Kazhamiaka, F. Abuzaied, P. Kraft, A. Agrawal, S. Kandula, S. Boyd, and M. Zaharia, "Solving large-scale granular resource allocation problems efficiently with POP," in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*, 2021, pp. 521–537.
  - [22] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar, "Learning to route," in *Proceedings of the 16th ACM Workshop on Hot Topics in Networks (HotNets)*, 2017.
  - [23] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Foundations and Trends in Machine Learning*, vol. 3, no. 1, 2011.
  - [24] X. Liu, S. Zhao, Y. Cui, and X. Wang, "FIGRET: Fine-grained robustness-enhanced traffic engineering," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 117–135.
  - [25] K. Rusek, J. Suárez-Varela, A. Mestres, P. Barlet-Ros, and A. Cabellos-Aparicio, "Unveiling the potential of graph neural networks for network modeling and optimization in SDN," in *Proceedings of the 2019 ACM Symposium on SDN Research (SOSR)*, 2019, pp. 140–151.
  - [26] A. A. AlQiam, Y. Yao, Z. Wang, S. S. Ahuja, Y. Zhang, S. G. Rao, B. Ribeiro, and M. Tawarmalani, "Transferable neural WAN TE for changing topologies," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 86–102.
  - [27] Y. Fan, J. Liu, P. Xie, and L. Liu, "Aether: Toward generalized traffic engineering with elastic multi-agent graph transformers," in *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, 2025, pp. 1–10.
  - [28] M. Ye, J. Zhang, Z. Guo, and H. J. Chao, "Path-based graph neural network for robust and resilient routing in distributed traffic engineering," *IEEE Journal on Selected Areas in Communications*, vol. 43, no. 2, pp. 422–436, 2025.
  - [29] D. Wu, X. Wang, Y. Qiao, Z. Wang, J. Jiang, S. Cui, and F. Wang, "NetLLM: Adapting large language models for networking," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 661–678.
  - [30] Z. Lai, H. Li, and J. Li, "StarPerf: Characterizing network performance for emerging mega-constellations," in *Proceedings of the 28th IEEE International Conference on Network Protocols (ICNP)*, 2020.
  - [31] E. Ekici, I. F. Akyildiz, and M. D. Bender, "A distributed routing algorithm for datagram traffic in LEO satellite networks," *IEEE/ACM Transactions on Networking*, vol. 9, no. 2, pp. 137–147, 2001.
  - [32] Y. Lu, F. Sun, and Y. Zhao, "Virtual topology for LEO satellite networks based on earth-fixed footprint mode," *IEEE Communications Letters*, vol. 17, no. 2, pp. 357–360, 2013.
  - [33] M. Hu, M. Xiao, W. Xu, T. Deng, Y. Dong, and K. Peng, "Traffic engineering for software-defined LEO constellations," *IEEE Transactions on Network and Service Management*, vol. 19, no. 4, pp. 5090–5103, 2022.
  - [34] L. Wei, T. T. Le, Y. Ji, M. Wang, Y. Liu, Y. Wang, and J. C. S. Lui, "Towards efficient traffic engineering via distributed optimization in large-scale LEO constellation," in *2024 IEEE International Conference on Communications Workshops (ICC Workshops)*, 2024, pp. 353–358.
  - [35] X. Liu, Z. Zhang, X. Qin, H. Zhou, and L. Zhao, "FlexSATE: Flexible and distributed traffic engineering with supervised learning in ultra-dense low-earth-orbit satellite networks," in *GLOBECOM 2024 - 2024 IEEE Global Communications Conference*, 2024, pp. 4466–4471.
  - [36] Z. Yang, G. Fan, A. Cao, Y. Guo, W. Li, C. Ying, T. Liu, S. Yue, and H. Jin, "Scalable traffic allocation in dynamic networks via end-to-end imitation learning," *IEEE/ACM Transactions on Networking*, vol. 34, 2026.
  - [37] Z. Yang, G. Fan, A. Cao, Y. Guo, W. Li, T. Liu, and H. Jin, "Learning to accelerate traffic allocation over large-scale networks," in *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, 2025.
  - [38] C. Eistan and G. Varghese, "New directions in traffic measurement and accounting," in *Proceedings of the ACM SIGCOMM 2002 Conference*, 2002, pp. 323–336.
  - [39] Z. Liu, A. Manousis, G. Vorsanger, V. Sekar, and V. Braverman, "One sketch to rule them all: Rethinking network flow monitoring with UnivMon," in *Proceedings of the ACM SIGCOMM 2016 Conference*, 2016, pp. 101–114.
  - [40] E. J. Candès and B. Recht, "Exact matrix completion via convex optimization," *Foundations of Computational Mathematics*, vol. 9, no. 6, pp. 717–772, 2009.
  - [41] J. You, X. Ma, D. Y. Ding, M. J. Kochenderfer, and J. Leskovec, "Handling missing data with graph representation learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
  - [42] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual multi-agent policy gradients," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
  - [43] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2019, pp. 4171–4186.
  - [44] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017.
  - [45] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations (ICLR)*, 2018.
  - [46] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 1024–1034.
  - [47] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Póczos, R. Salakhutdinov, and A. J. Smola, "Deep sets," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 3391–3401.
  - [48] G. Giuliarì, T. Klenze, M. Legner, D. Basin, A. Perrig, and A. Singla, "Internet backbones in space," *ACM SIGCOMM Computer Communication Review*, vol. 50, no. 1, 2020.
  - [49] G. Giuliarì, T. Ciussani, A. Perrig, and A. Singla, "ICARUS: Attacking low earth orbit satellite networks," in *Proceedings of the 2021 USENIX Annual Technical Conference (USENIX ATC)*, 2021, pp. 317–331.
  - [50] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.